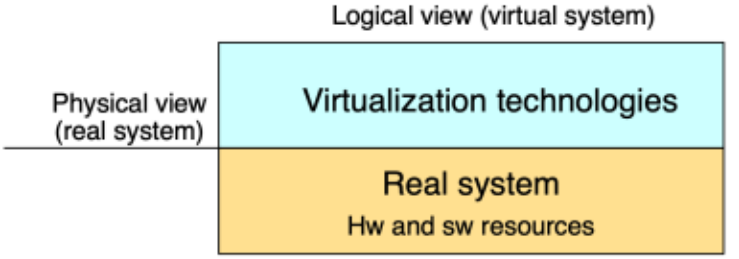
Virtualization

- Provides a high-level abstraction that hided the details of the underlying implementation
  - Hardware and software resources
- Offers a logical view of computing resources that differs from the physical ones



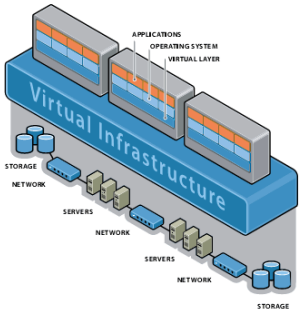
- How? Decouples user-perceived architecture and behavior of hw and sw resources from their physical implementation
- Goals:
  - Portability, efficiency, reliability, security, …

Virtualization of resources

- System virtualization
  - Virtualizes hw and sw resources
  - Virtual machines, containers, unikernels, …

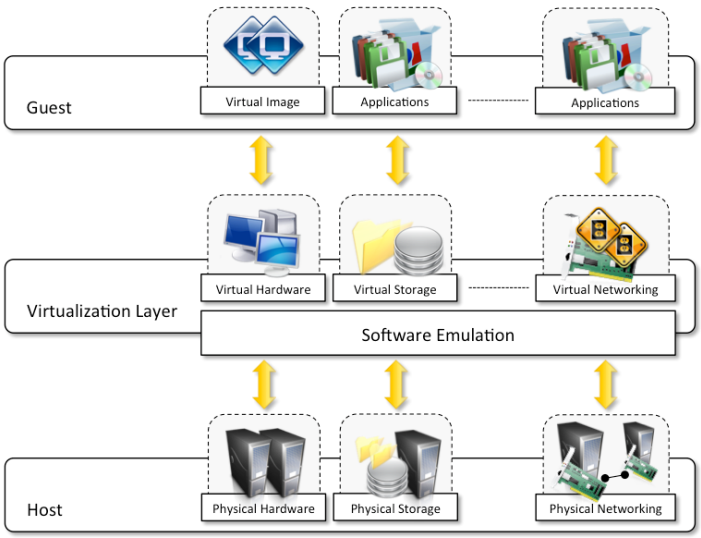
Our focus

- Storage virtualization
  - Abstracts storage resources
  - Storage Area Network (SAN), virtual disks, …
- Network virtualization
  - Virtual LAN (VLAN), Virtual Private Network (VPN), …
- Data center virtualization
  - Software-Defined Data Center (SDDC)



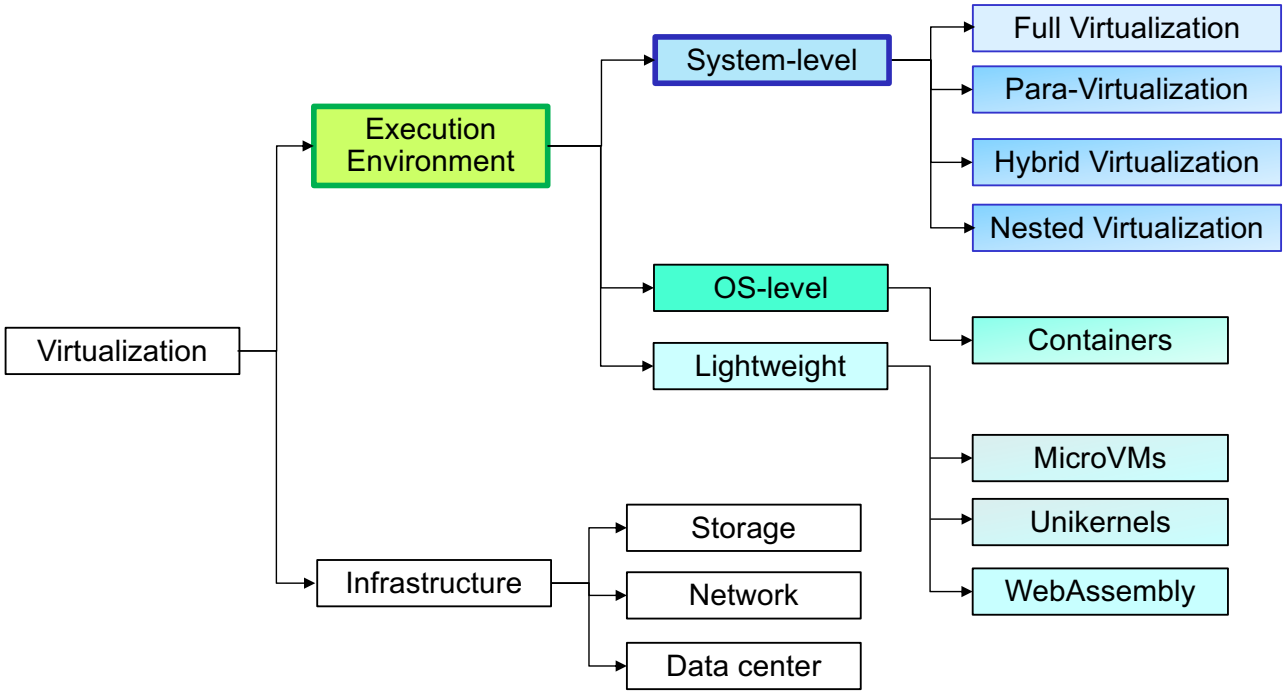
Major components of virtualization

- **Guest**:
  - The virtualized environment
  - Interacts with the virtualization layer, not directly with host
- **Host**:
  - The physical system
  - Provides the original hw and sw resources



- **Virtualization layer**
  - Responsible for creating the virtual environment
  - Manages resource allocation, isolation, and interaction between guest and host

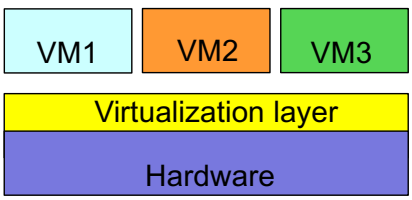
Taxonomy of virtualization techniques



- **Execution environment virtualization:** the oldest, most popular and most developed area ⇒ our focus

Virtual Machine

- **Virtual Machine** (**VM**): a complete compute environment with its own isolated processing capabilities, memory, and communication channels
- Represents hw/sw resources of a physical machine differently from their physical reality
  - E.g., VM hw resources (CPU, network card, ...) can differ from physical machine's hw resources
  - E.g., VM sw resources (OS, applications) can differ from physical machine's sw resources
- A single physical machine can host multiple independent VMs



Virtualization: a brief history

- An “old” idea in computer science
  - Originates in the 1960s in centralized computing
  - Goals: let legacy software run on expensive mainframes and transparently share scarce physical resources
  - Example: IBM System/360-67 mainframe



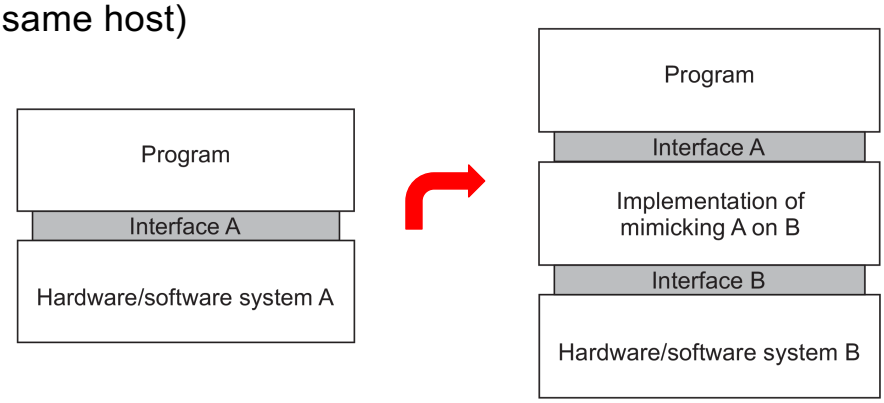
- Shift in the 1980s with the rise of PCs
  - OS multitasking to transparently share computing resources
  - Virtualization became less relevant for a while

Virtualization: a brief history

- Late 1990s: virtualization interest returns
  - Needed to simplify programming on special-purpose hardware
  - VMware founded in 1998
- Why the comeback?
  - Hardware evolves faster than software → higher management costs
  - Poor application portability
  - Significant under-utilization of hardware resources
  - Need to share resources efficiently and reduce infrastructure costs
- Today, virtualization is a core technology in cloud computing

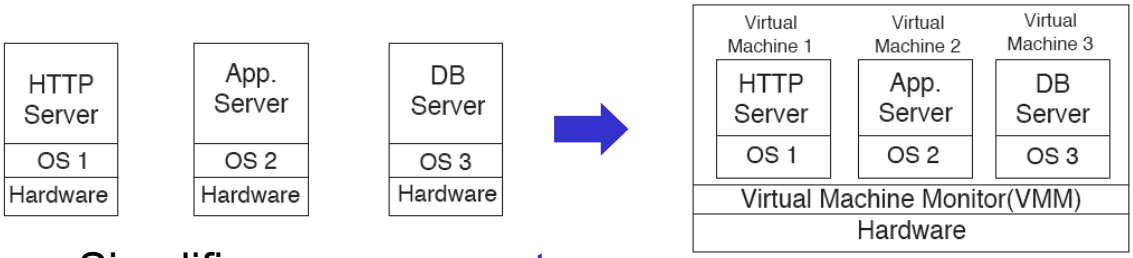
Virtualization: benefits

- Improves portability, compatibility, interoperability, and migration
  - Hardware independence: create once, run everywhere
  - Legacy support: run old OSes/apps on modern hardware
  - Support heterogeneous environments (different OSes on same host)



Virtualization: benefits

- Enables server consolidation and efficiency
  - VMs share one physical machine (multiplexing)
  - Reduces number of servers → lower cost, energy, and space usage
  - Better utilization of otherwise idle hardware



- Simplifies management
  - Easier maintenance and upgrades
  - Rapid provisioning: quickly create, clone, or destroy VMs
  - Snapshots and rollback simplify testing and recovery
  - Per-VM monitoring and resource accounting (important for cloud billing)

Virtualization: benefits

- Improves reliability and security
  - Strong isolation: faults or attacks in one VM do not affect others
  - Malware, bugs, and crashes remain contained within the VM
  - Micro-segmentation and secure virtual networks
  - Disaster recovery: VM images can be replicated and restored elsewhere
- Provides performance isolation
  - Controlled scheduling of shared resources ensures predictable performance
  - Reduces noisy-neighbor effects

Virtualization: benefits

- Improves load balancing and high availability
  - Improves load distribution across physical hosts
  - Live migration enables moving VMs seamlessly across physical hosts and reduces downtime
  - Enhances availability in cloud and data-center environments
- Better development and testing
  - Safe sandbox environments
  - Easy creation of testbeds and multi-VM setups
  - Useful for CI/CD pipelines and experimentation

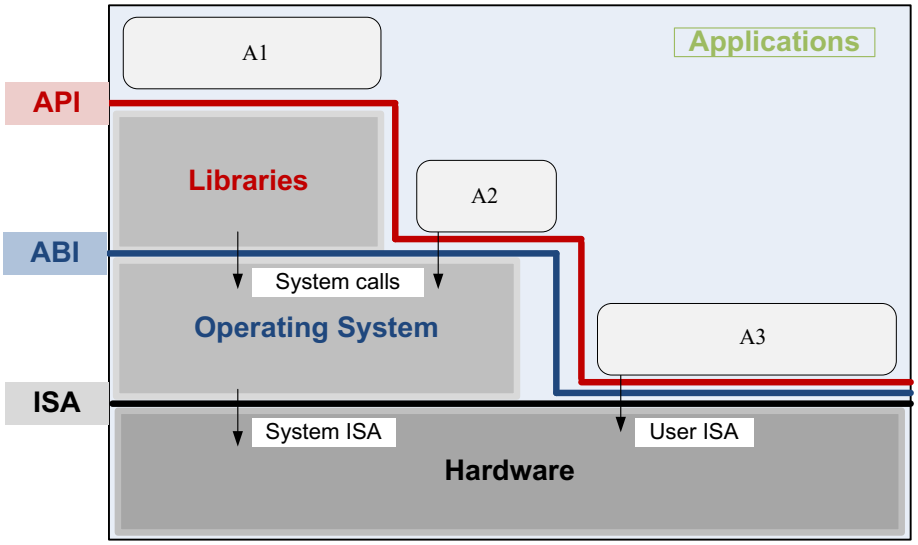
Reasons to use virtualization

- Personal and educational
  - Learn and experiment with multiple OSes
  - Develop, test, and debug applications in isolated environments
  - Simulate distributed systems on a single machine

- Enterprise and professional
  - Support DevOps workflows and continuous integration
  - Encapsulate entire systems in VM images for replication, migration, or redeployment
  - Ensure business continuity and disaster recovery readiness
  - Run legacy applications or OSes on modern platforms

Un computer offre generalmente 4 tipi diversi di interfacce su 3 livelli diversi: un interfaccia tra hardware e software ovvero l'instruction set architecture (ISA), che forma l'insieme delle istruzioni macchina che si suddivide in istruzioni privilegiate e istruzioni generiche. L'Application binary interface (ABI) che fornisce le system call come interfaccia con in sistema operativo. E un'interfaccia per le funzioni di libreria ovvero l'application programming interface (API)

Interfaces in computer system



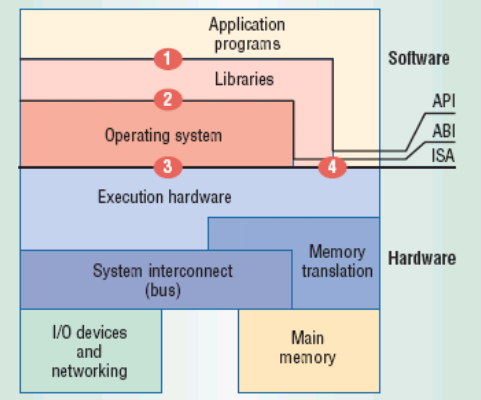
Applications:

- use library functions (A1)
- make system calls (A2)
- execute machine instructions (A3)

Interfaces in computer system and virtualization

Levels where virtualization can be realized

- Key idea: virtualization is strictly related to computer system interfaces → mimics their behavior
  - Hw/sw interface (**system ISA**): manages system resources, privileged instructions executed only by OS [interface 3]
  - Hw/sw interface (**user-level ISA**): focused on computation, *unprivileged* instructions executed by any program [interface 4]
  - System calls: controlled access to OS services from user applications [interface 2]
  - **ABI** (Application Binary Interface): ensures binary compatibility [interfaces 2 + 4]
  - Library calls (**API**): high-level interface for application software to use system services [interface 1]



Smith and Nair, The architecture of virtual machines, *IEEE Computers*, 2005

Implementation levels of virtualization

- Virtualization can be implemented at various operational levels
  - Primary involved interface is indicated within [ ]
- ISA level [interface 4]
- **Hardware level** (aka *system VMs*) [interface 3]
- **Operating system level** (aka *containers*) [interface 2] ← Our main focus
- Library level [interface 1]
- User application level (aka *process VMs*) [interfaces 4 + 1]

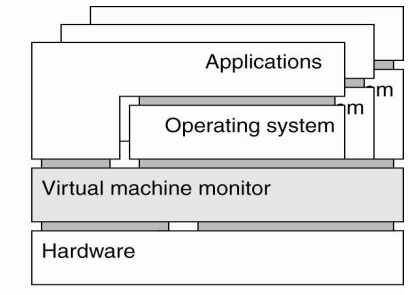
Implementation levels of virtualization

- ISA level
  - Goal: emulate a given ISA (unprivileged instructions) on the host machine for portability
  - Methods
    - Code interpretation: interpret each source instruction to host ISA instructions
    - Dynamic binary translation: convert code in blocks for faster execution
  - Example: run MIPS binary code on x86 host

Implementation levels of virtualization

- Hardware level (aka *system VMs*)
  - Goal: virtualize host resources (CPUs, memory, I/O) via **Virtual Machine Monitor** (**VMM**), aka *hypervisor*
    - VMM manages hw resources and shares them among multiple VMs
    - VMM intercepts privileged instructions and enforces isolation
  - Provides complete environment where multiple VMs coexist
  - Examples: VMware, KVM, Xen, Parallels, VirtualBox
  - Examples: microVMs
    - Lightweight VMs with minimal OS (e.g., Firecracker, Kata Containers)

Multiple instances of combinations <applications, OS>



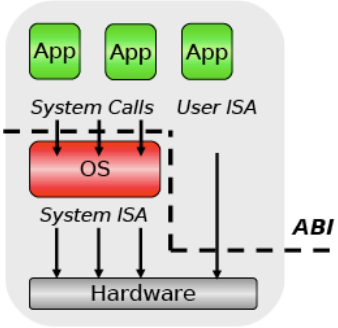
Implementation levels of virtualization

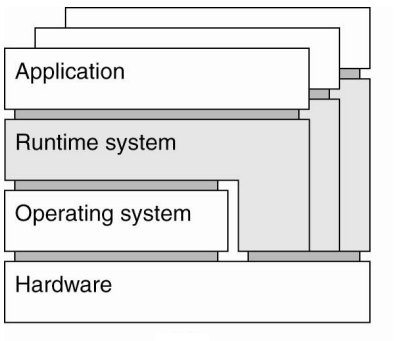
- Operating system level (aka *containers*)
  - Goal: create multiple isolated containers sharing the OS kernel
  - Containers rely on OS kernel to provide system calls
  - Examples: Docker, LXC, Podman
- Library level
  - Goal: run apps on host OS that does not natively support them
    - Translate high-level API calls from the application and translate them to host OS API rather than emulating full OS
  - Examples:
    - Wine: Windows apps on top of Linux https://www.winehq.org
    - Cygwin: "Get that Linux feeling – on Windows" https://cygwin.com

Implementation levels of virtualization

- User application level (aka *process VMs*)
  - Virtual platform for a single process
  - Provides virtual ABI/API to user app
  - Executes portable intermediate code (e.g., Java bytecode)
  - Examples: JVM, .NET CLR, WebAssembly runtimes

Multiple instances of combinations <application, runtime system>





System-level virtualization: terminology

- Let's focus on **system-level virtualization** via VMM/hypervisor

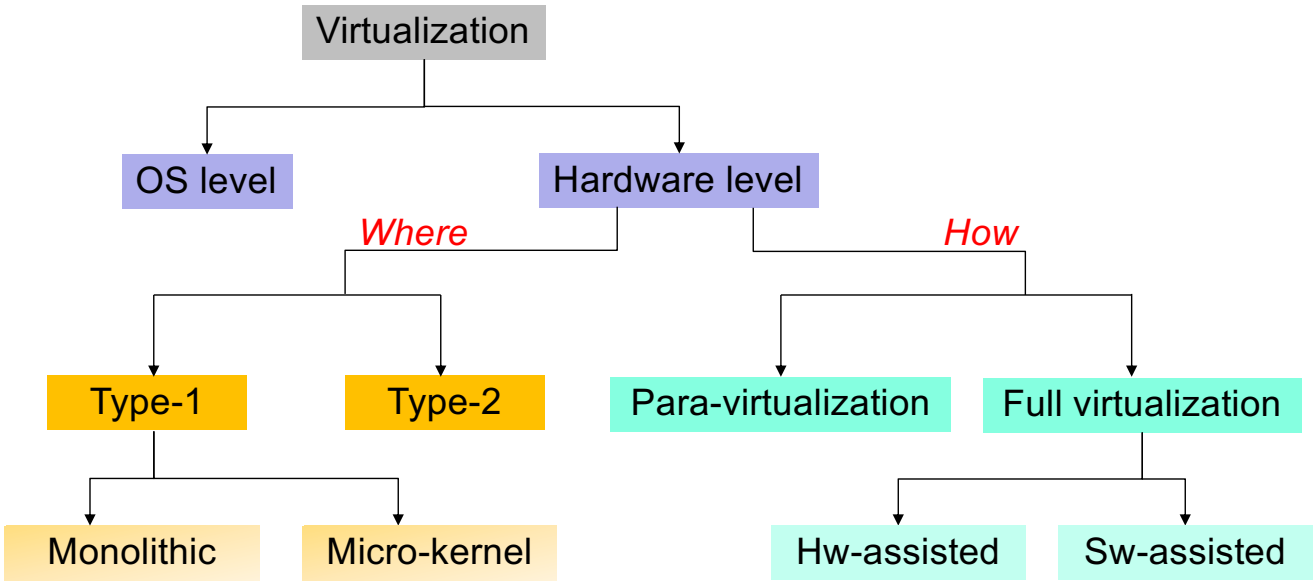
- **Host**: base platform on top of which VMs run; consisting of:
  - Physical machine
  - *Optional* host OS
  - VMM/hypervisor
- **Guest**: everything inside the VM
  - Guest OS
  - Applications executed inside the VM

System-level virtualization: taxonomy

- System-level virtualization classification
  1. Where VMM is deployed
     - **System VMM**
     - **Hosted VMM**
  2. How to virtualize instruction execution
     - **Full virtualization**
       - *Software-assisted*
       - *Hardware-assisted*
     - **Paravirtualization**

studiamo 3 soluzioni per virtualizzare l'esecuzione delle istruzioni

System-level virtualization: taxonomy

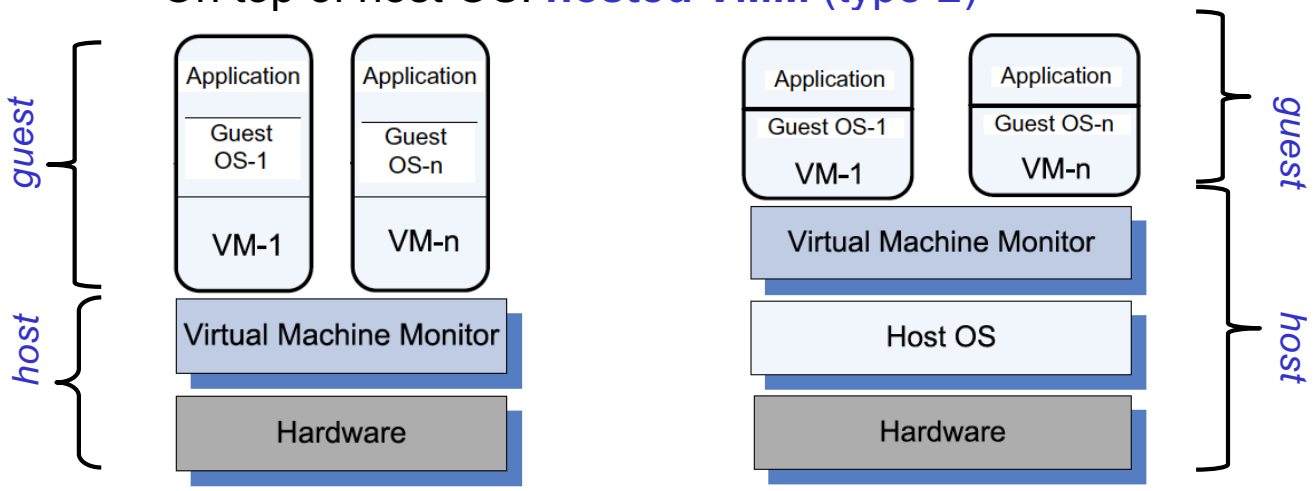


Comparison of platform virtualization software
https://en.wikipedia.org/wiki/Comparison_of_platform_virtualization_software

System vs hosted VMM

VMM deployment level in system architecture

- Directly on hardware: **system VMM** (type-1 / native / bare-metal)
- On top of host OS: **hosted VMM** (type-2)



System VMM Hosted VMM

System vs hosted VMM

- **System VMM** (*type-1*): runs directly on hw and provides virtualization features integrated into a minimal OS
  - Can use either *microkernel* (only basic functions, no device drivers) or *monolithic* (integrated drivers) architecture design
  - Examples: Xen, VMware ESXi, Xen, Microsoft Hyper-V, KVM, Nutanix AHV
- **Hosted VMM** (*type-2*): runs on top of host OS and uses host OS drivers and system calls
  - Interacts with the host OS via the ABI and presents virtualized hw (emulated or hardware-assisted) to the guest OSes
  - ✓ Leverages host OS for device management and low-level services (e.g., resource scheduling)
  - ✓ No guest OS modifications required
  - ✗ Lower performance with respect to system VMM due to extra sw layer
  - Examples: VirtualBox, Parallels Desktop

Privileged instructions

- Instructions that can only be executed by OS kernel (or hypervisor) nel nostro caso dal hypervisor
- They access or control critical system resources such as:
  - CPU modes (switching between user and kernel)
  - Memory management (page tables, MMU)
  - I/O devices (disk, network, peripherals)
  - Interrupt handling

sarà importante virtualizzare queste istruzioni. N.B. siamo nel caso in cui il VMM è sul bare-metal e non on top del host OS. in quel caso l'host OS si occupa delle istruzioni a livello di kernel

- Purpose: prevent user programs from directly manipulating hw or compromise system stability
- Examples:
  - Changing the CPU mode (user → kernel)
  - Modifying control registers (x86 `CR3` for page tables)
  - Direct access to I/O ports

Full virtualization vs paravirtualization

How to manage execution of privileged instructions in VMs

- **Full virtualization** da l'illusione di non avere alcun intermediario
- **Paravirtualization** il guest OS è opportunamente modificate per eseguire nel contesto di virtualizzazione

- Full virtualization
  - VMM exposes to each VM simulated hw interfaces *identical* to the underlying physical machine l'hypervisor deve possedere un meccanismo trasparente per eseguire istruzioni kernel level
  - VMM intercepts attempts to perform privileged instructions (e.g., I/O, TLB update) and emulates their behavior
  - Examples: KVM, VMware ESXi, Microsoft Hyper-V
- Paravirtualization
  - VMM exposes to each VM simulated hw interfaces that are *similar* but not identical to the underlying physical machine
  - Hardware is not fully emulated; a minimal software layer (Virtual Hardware API) manages VMs and ensures isolation
  - Examples: Xen, Oracle VM, PikeOS

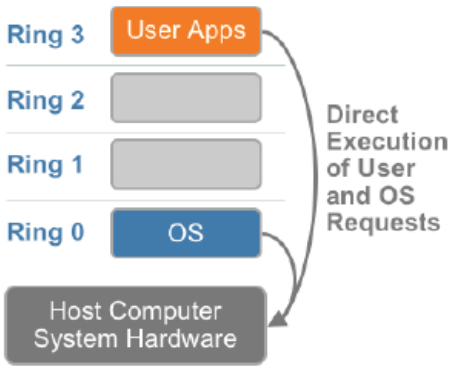
Full virtualization vs paravirtualization

- Full virtualization: pros
  - ✓ Runs unmodified guest OSs
  - ✓ Strong isolation between VMs: improves security and allows emulating different architectures independently
- Full virtualization: cons
  - ✗ VMM is more complex
    - It must intercept and emulates all privileged instructions
  - ✗ Requires processor support to make virtualization more efficient: why?

Issues to address for system-level virtualization

- Non-virtualized processor architectures use at least 2 protection rings: supervisor and user
  - Ring 0: most privileged (unrestricted access to system resources)
  - Ring 3: least privileged

x86 architecture w/o virtualization



vediamo le differenze tra un processore non virtualizzato ed uno virtualizzato

- With virtualization
  - VMM operates in supervisor mode (ring 0)
  - Guest OS and applications run in less privileged rings (guest OS in ring 1 or 3)
  - **Ring deprivileging** problem: guest OS executes in a ring that is not its own → cannot execute privileged instructions directly
  - **Ring compression** problem: since both guest OS and applications run at the same level, guest OS must enforce protection

più importante →

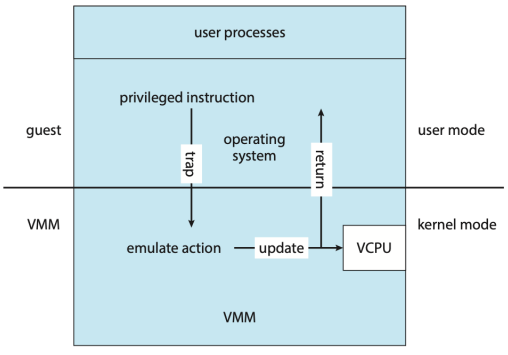
dovrebbe essere nel ring 0 ma non lo è

di fatti servono meccanismi aggiuntivi per il guest OS

Addressing ring deprivileging

- **Trap-and-emulate**
  - When the guest OS executes a privileged instruction (e.g., `lidt`), an exception (trap) is raised and control is transferred to the VMM; the VMM performs a safety check on the requested operation, executes ("emulates") its behavior, and returns the result to guest OS
  - Non-privileged instructions, instead, run by guest OS do not trap and are executed directly

se ad esempio il guest OS vuole aggiornare la tabella degli interrupt, la system call avvisa l'hypervisor che la intrappola e si fa carico di eseguire l'istruzione virtualizzata e di ritornare il risultato al guest OS



- Issue: on some architectures, certain instructions do not trap, requiring more complex solutions

Popek and Goldberg virtualization requirements

- Popek and Goldberg requirements (1974) define when an architecture can support *efficient* virtualization
- Conditions:

Equivalence
producono istante per istante lo stesso risultato

Resource control

Efficiency

1. A program running under the hypervisor should exhibit a behavior essentially identical to that demonstrated when running on an equivalent machine directly.

2. The hypervisor should be in complete control of the virtualized resources.

3. A statistically significant fraction of machine instructions must be executed without the intervention of the hypervisor.

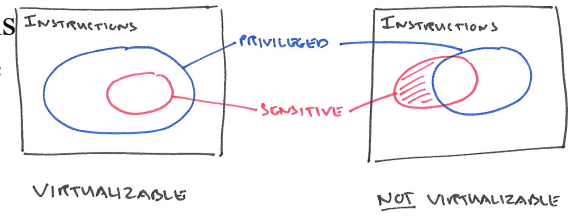
Formal requirements for virtualizable third generation architectures, 1974
https://dl.acm.org/doi/pdf/10.1145/361011.361073

sulla base di questi requisiti possiamo classificare le istruzioni ISA

Instruction types and virtualization theorem

- ISA instruction classification
  - Privileged instructions: execute only in supervisor mode; trap in user mode
  - Sensitive instructions: affect or reveal CPU/system state classe importante
    - Control-sensitive: modify CPU configuration/state (e.g., interrupt/paging tables)
    - Behavior-sensitive: reveal CPU state
  - Innocuous instructions: neither sensitive nor privileged
- **Popek and Goldberg theorem**: *a VMM can be constructed if all sensitive instructions are privileged*

dunque le sensitive instructions devono essere un sottoinsieme delle istruzioni privilegiate



c'è solo un intersezione quindi non è virtualizzabile

https://blog.acolyer.org/2016/02/19/formal-requirements-for-virtualizable-third-generation-architectures

Problem and implications

- Problem: some architectures have sensitive non-privileged instructions, e.g.:
  - x86: many sensitive non-privileged, such as:
    - `pushf`, which pushes the flags register on the stack
  - MIPS: mostly virtualizable, but `$k0`, `$k1` registers are accessible in user mode
  - ARM: mostly virtualizable, but some instructions undefined in user mode
- Must virtualize:
  - Privileged instructions → trap to VMM and emulate
  - Sensitive non-privileged instructions → must also be virtualized

pushf è push del registro dei flag sullo stack

ma sono i registri utilizzati per salvare e caricare lo stato

Solutions for virtualizing sensitive instructions

- Trap-and-emulate
  - Privileged instructions trap → VMM emulates instruction behavior
  - Non-privileged instructions run directly on CPU
  - Limitation: some sensitive non-privileged instructions do not trap on certain CPUs → which solutions?

1. **Hardware-assisted virtualization** (hw level)
   - VMM relies on hardware to intercept privileged and sensitive instructions automatically
   - All sensitive instructions trap automatically to VMM
2. **Fast binary translation** (sw level)
   - VMM rewrites sensitive instructions on the fly
3. **Paravirtualization**
   - Modify guest OS to avoid or neutralize sensitive non-privileged instructions

abbiamo il supporto del hw per la virtualizzazione

soluzione disponibile dal 2005 e oggigiorno la più usata

prima soluzione ideata a fine anni 90

l'overhead era però elevato

soluzione ideata nei primi anni 2000

realizzata per la prima volta a Cambridge per l'hypervisor Xen, per ridurre l'overhead della fast binary translation

Hardware-assisted CPU virtualization

- Idea: introduce a new privilege level for the hypervisor
  - Hypervisor privilege > OS/kernel privilege
  - All sensitive instruction trap to hypervisor level
    avviene quindi una trap per le sensitive instruction
- Provides two new CPU operating modes
  - **Root mode** for the VMM
  - **Non-root mode** for guest OSs
- Each mode supports all 4 x86 protection rings

grazie a queste modalità si possono utilizzare appunto tutti e 4 i ring, risolvendo il problema del ring deprivileging e del ring compression

Hardware-assisted CPU virtualization

- VMM runs in Root-Ring 0, while guest OSs run in guest mode in Non-Root Ring 0 at their original privilege levels
  - Eliminates ring deprivileging and ring compression issues

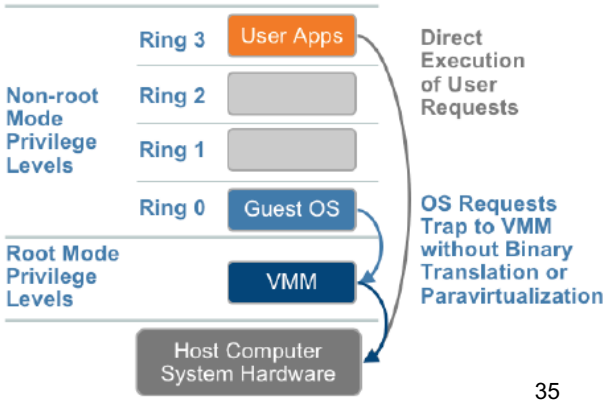
la trap è implementata a livello hardware

- When a guest OS executes a privileged or sensitive instruction in Non-Root mode, the CPU automatically stops guest execution (VM exit) and switches control to the VMM, which runs in Root mode
  - VMM can control guest execution through VM control data structures in memory

VM-exit è simile ad un context switch. il contesto viene salvato prima di cambiare mode

- Implemented in modern CPUs (e.g., Intel VT-x and AMD-V, since 2005)

x86 architecture with with full virtualization and *hardware-assisted CPU virtualization*



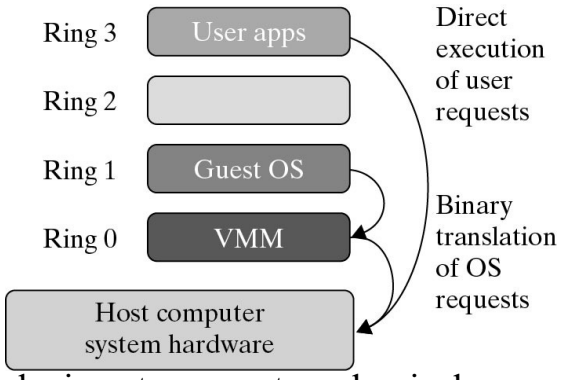
Fast binary translation

- Software-based and the oldest solution
- VMM scans code before execution
- Replaces blocks containing privileged instructions with functionally equivalent blocks that notify the VMM when executed
  - Translated blocks run directly on hw and are cached for future reuse

la trap in questo caso è implementata a livello software

✗ Higher complexity and lower performance compared to hw-assisted virtualization

x86 architecture with full virtualization and fast binary translation



soluzione trasparente e che risolve il ring decompression

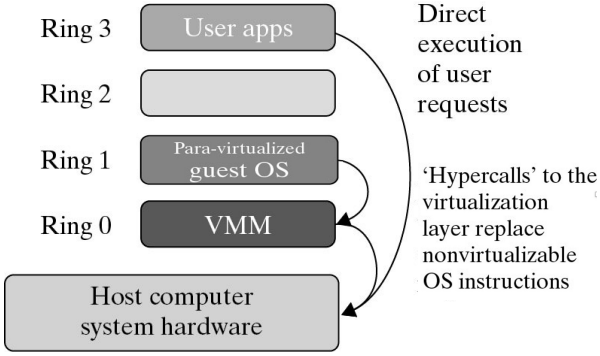
Paravirtualization

- Non-transparent virtualization
  - Guest OS kernel must be modified to use the virtual API exposed by the VMM
- Non-virtualizable instructions are replaced with *hypercalls* that communicate directly with the hypervisor
- Hypercall: *software trap* from the guest OS to the hypervisor

ecco la modifica del guest OS

hypercall : hypervisor = syscall : kernel

x86 architecture with *paravirtualization*



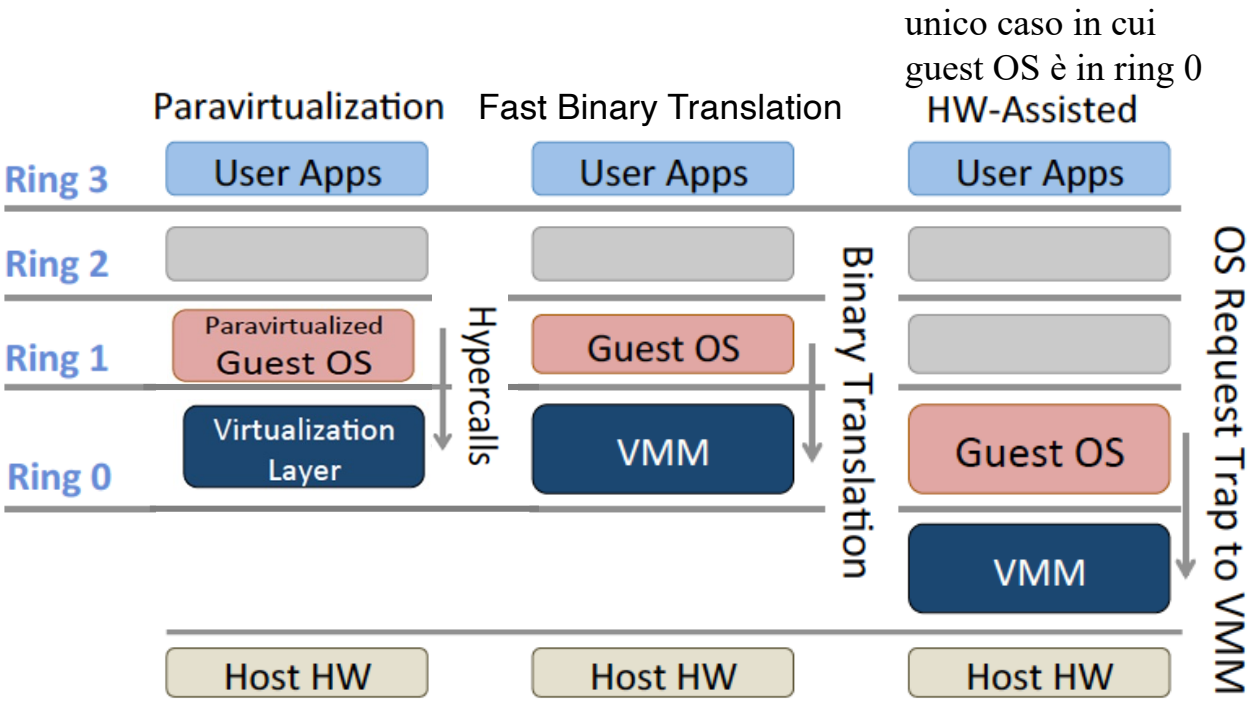
- Hypercall execution
  - Control jumps from guest OS to hypervisor
  - Hypervisor performs the requested operation
  - After handling, control returns to guest OS kernel, which then resumes the application

Paravirtualization

- Pros:
  - ✓ Easier and more practical to implement than full virtualization on non-virtualizable hardware
  - ✓ Lower runtime overhead compared to fast binary translation
  - ✓ Does not require virtualization extensions (unlike hw-assisted virtualization)

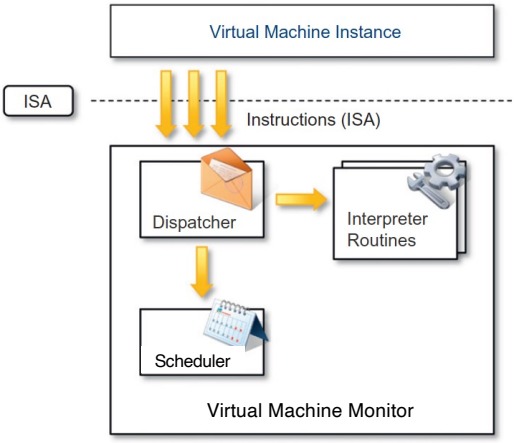
- Cons:
  - ✗ Requires source code access to modify the guest OS and make it paravirtualized windows non approva
  - ✗ Maintenance overhead: each paravirtualized OS version must be maintained separately
    - Paravirtualized OS cannot run directly on hardware

Summing up different approaches



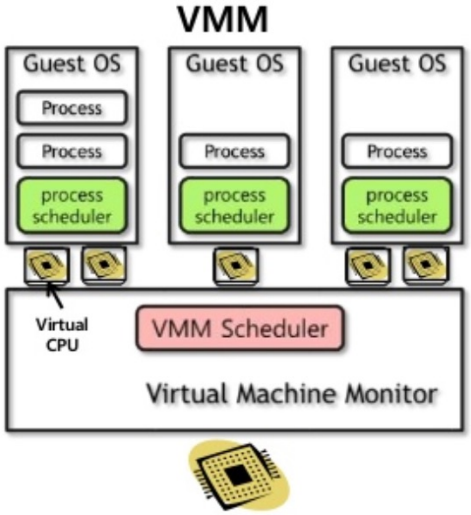
VMM reference architecture

- Main modules
  - *Dispatcher*: VMM entry point that reroutes privileged instructions issued by VMs to the appropriate module
  - *Scheduler*: invoked by the dispatcher when a VM requests system resources; decides resource allocation among multiple VMs componente fondamentale
  - *Interpreter*: executes the appropriate routine for each privileged instruction



VMM CPU scheduler

- Guest OS sees virtual CPUs
- Physical CPUs on host machine are multiplexed among VMs
- How to schedule virtual CPUs on physical CPUs?



Case study: Xen

- What Xen is https://www.xenproject.org
  - Open-source type-1 (system VMM) hypervisor with microkernel design
  - Originated at Cambridge University in the late 1990s
  - Foundation for XenServer, Oracle VM, and used in cloud and embedded systems (e.g., AWS, Alibaba, ARM)
  - Supports paravirtualization (**PV**), hardware-assisted virtualization (**HVM**) and PV-HVM hybrids
- Key ideas for paravirtualization (no hw support yet)
  - Guest OS interacts with the hypervisor via a **hypercall API**
  - Paravirtualization replaces:
    - Privileged instructions → hypercalls
    - Page table updates
    - I/O device access
    - Interrupt & timer operations
  - Requires PV-enabled OSs and drivers (e.g., Linux)

prodotto che ha posto le fondamentale per gli hypervisor moderni

appunto bare metal

oggi AWS usa il suo VMM basato su KVM

usato da AWS fino al 2017.

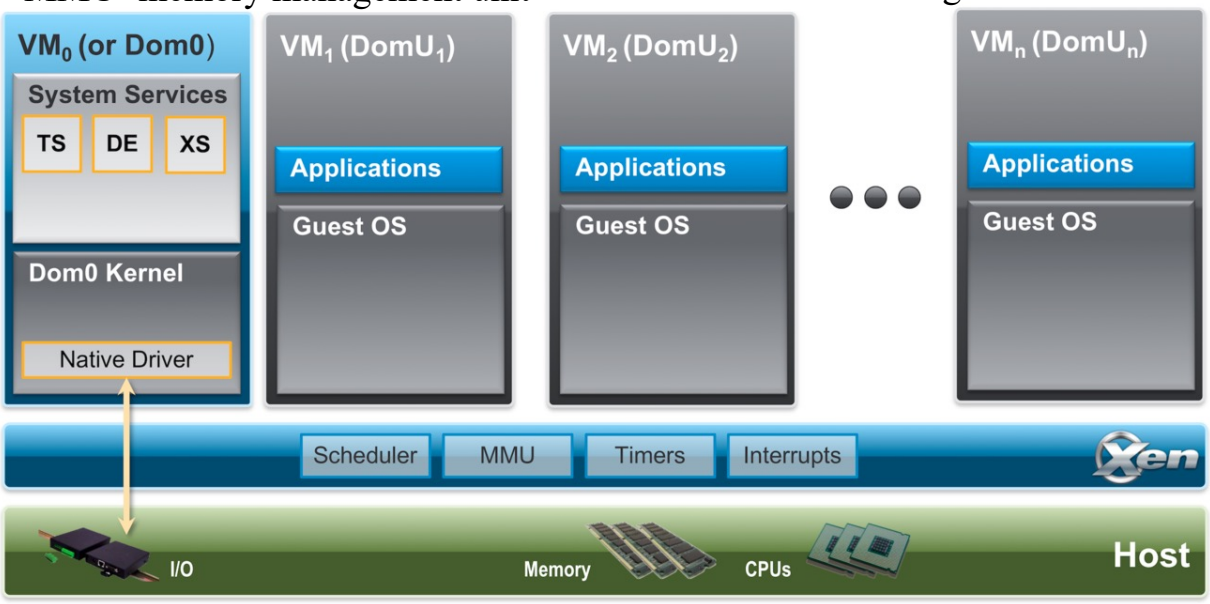
repac utili di cosa si fa carico la paravirtualizzazione

Xen: architecture

andiamo ad osservare nel dettaglio i componenti dell'architettura di Xen

MMU=memory management unit

da notare che Xen non gestisce l'I/O



https://wiki.xenproject.org/wiki/Xen_Project_Software_Overview

la VM Dom0, mette a disposizione dei driver native e dei servizi di sistema per gestire l'I/O in Xen.

questa separazione è presente anche in soluzioni PV-HVM hybryd

Xen: architecture

- Thin, micro-kernel hypervisor
  - 300K LoC on x86, 65K on ARM
- Hypervisor manages vCPU scheduling, memory, timers, and interrupts
- **DomU** (unprivileged guest domains / VMs)
  - Run guest OSs
  - Have no privilege to access hardware or I/O functionality
- **Dom0** (privileged control domain)
  - Contains the device drivers and multiplexes I/O for DomU
  - Manages direct hardware access
  - Runs management services
    - XenStore (XS): shared key–value config store
    - Toolstack (TS): VM lifecycle management and configuration
    - Device Emulation (DE)
  - Mandatory, started on boot

linee di codice

XenStore contiene TUTTI i metadati del contesto virtualizzato. è capace anche di notificare le VM i cambiamenti di configurazioni

la migrazione avviene anche live

servizio che interagisce con i driver per la gestione del I/O

il disaccopiamento Xen e Dom0 consente di limitare il numero di righe di codice di Xen, riducendo la superficie d'attacco possibile

Xen and paravirtualization

- In modern deployments Xen no longer uses PV to virtualize the CPU, HVM is preferred
  - PV guests are still used when hardware lacks virtualization extensions
- Paravirtualization remains important for I/O
  - Xen uses virtual-only devices with PV drivers to avoid slow hardware emulation, enabling faster and more efficient I/O

i dettagli non saranno trattati

https://wiki.xenproject.org/wiki/Understanding_the_Virtualization_Spectrum

- Summarizing Xen impact
  - First hypervisor to popularize paravirtualization
  - Influenced modern cloud hypervisors (AWS, Google)
  - Clear example of a micro-hypervisor
    - Reduces attack surface, increases stability and security, and keeps the hypervisor small and lightweight
  - Still used in security-sensitive and embedded environments

Xen: Credit scheduler

- VMM scheduler selects which virtual CPUs (vCPUs) of all VMs run on the physical CPUs (pCPUs)
  - Adds a scheduling layer below the guest OS scheduler
- Default Xen scheduler: **Credit** scheduler
  https://wiki.xenproject.org/wiki/Credit_Scheduler
- Goals: alcuni condivisi con lo scheduling tradizionale di un OS
  - Proportional fair share: each domain receives a fair share of pCPU proportional to its weight ciascun Dom riceve una quota proporzionale al peso del Dom (anche Dom0)
  - Work-conserving: no pCPU stays idle if runnable vCPUs exist
  - Low latency: fast switching among runnable vCPUs
- Key parameters:
  - Weight: relative share of CPU time (default = 256) più importante. contatore di crediti decrementato con l'uso della CPU
  - Cap: optional limit on maximum CPU share parametro di configurazione
    - cap = 0 → vCPU can use extra CPU time (work-conserving) default value
    - cap > 0 → upper bound (% of one pCPU)

Xen: Credit scheduler

- Each vCPU receives credits proportional to its weight
- Running consumes credits → vCPU becomes:
  - UNDER (credits ≥ 0) il weight, con il passare del tempo viene resettato
  - OVER (credits < 0)
- Each pCPU keeps a runqueue of vCPUs
  - UNDER vCPUs first, then OVER
  - OVER vCPUs run only if no UNDER exist
  - Round-robin within each group della pCPU
  - Time slice: 30 ms
- Placement pCPU di riferimento
  - vCPU is inserted into its home pCPU runqueue when runnable
  - May be migrated to another pCPU by load balancing
- Load balancing favorisce sempre le vCPU UNDER
  - Before going idle, a pCPU checks others for runnable UNDER vCPUs; ensures no pCPU is idle while work exists

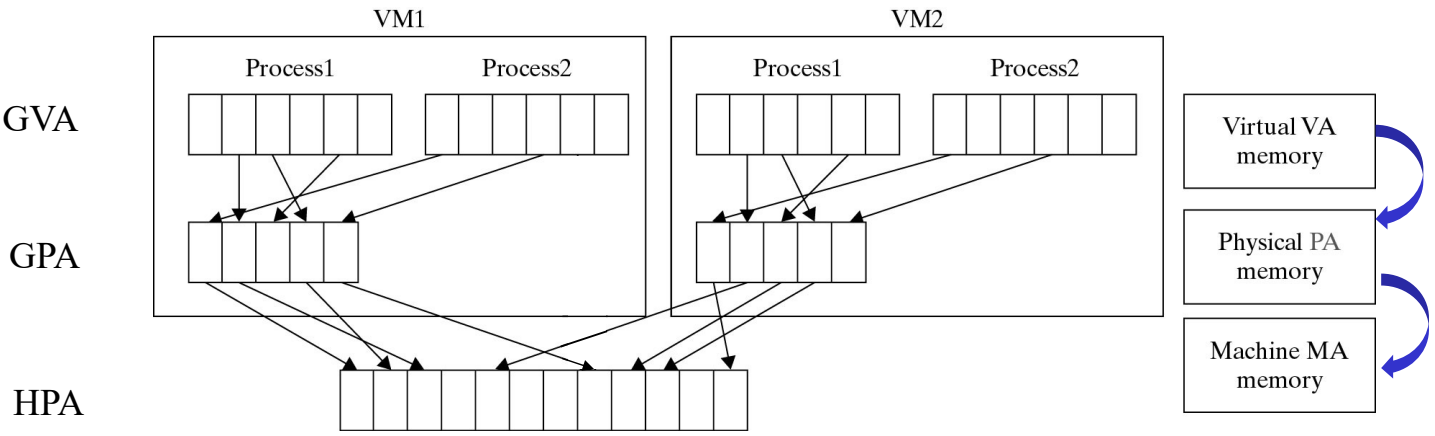
Memory virtualization gestita dalla MMU

- *Non-virtualized* environment
  - Single-level memory mapping: virtual address (VA) → physical address (PA) via page tables
  - MMU and TLB accelerate translations
- *Virtualized* environment
  - Multiple VMs share machine memory → VMM must partition it
  - Requires **two-level mapping:** guest virtual address (**GVA**) → guest physical address (**GPA**) → host physical address (**HPA**)

un OS si aspetta che lo spazio di memoria inizi da 0 ma se partiziono la memoria ciò non accade ecco perché 2 livelli di mapping

- Terminology
  - Guest virtual memory: visible to apps
  - Guest physical memory: visible to guest OS
  - Host physical memory: actual machine memory visible to VMM

Two-level memory mapping

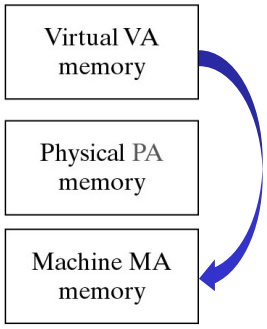


- **GVA → GPA → HPA**
- GPA ≠ HPA: why?
  - Because each guest OS assumes it owns *contiguous, zero-based* physical memory
  - VMM must preserve this illusion

meccanismo che evita il doppio processo di mapping

Shadow page tables (SPT) gestite da hypervisor

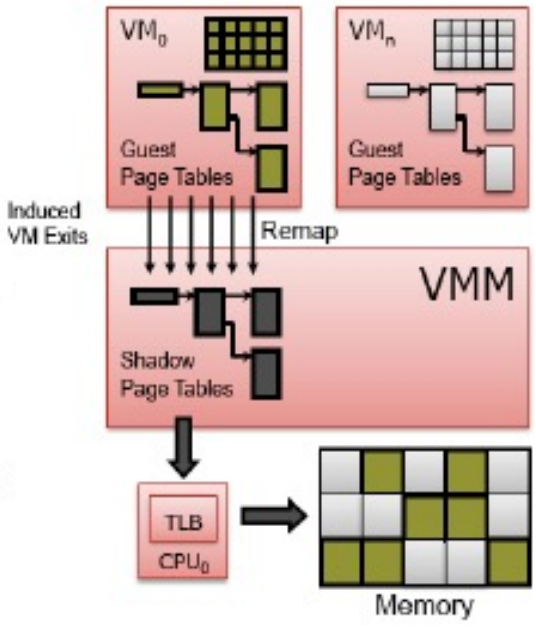
- Goal: avoid performing two translations on every access by giving hardware a direct **GVA → HPA** mapping
- Key mechanisms:
  - VMM maintains shadow page tables (**SPTs**) **which** directly maps GVA to HPA and uses them to accelerate
  - Guest OS builds its own page tables normally, but MMU uses SPTs created by the VMM
  - VMM keeps SPTs consistent with guest page tables
  - Guest page tables are mapped read-only
  - Any guest update → trap to VMM, which updates both guest PT and SPT ("memory tracing")



le page table del guest sono solo read only, il VMM gestisce la write sia SPT che page table del guest

SPT drawbacks

- High overhead: frequent traps and VM exits
  - Every guest PT write → VM exit → high overhead
  - VM exit: similar to a very expensive context switch
- Large memory footprint for maintaining SPTs
- Complex software logic
  - VMM intercepts paging operations and constructs copy of PTs



soluzione ai drawback

Hw support for memory virtualization

- More efficient hardware solution: Second Level Address Translation (SLAT)
- Hardware performs the second translation (GPA → HPA)
- Removes need for SPT maintenance
- Dramatically reduces traps and VM exits
- Provides significant speedup (~50%) for memory-intensive workloads
- Examples: Intel EPT, AMD RV

VM portability

- **VM image**: a single file for VM containing the bootable OS, data files, and applications
- VM images come in different formats dipendono dal hypervisor
- How to import and export VM images and avoid vendor lock-in? Soluzione, formato standard
- Open Virtualization Format (OVF)
  - Open industry standard (ISO 17203) for packaging and distributing VMs
    - Virtual-platform agnostic
  - Image stored in .ova file (Open Virtual Appliance)
  - VM configuration specified in XML format within a .ovf file
  - Supported by many hypervisors, including Hyper-V, VMware, VirtualBox, XenServer

VM resizing and migration

- Techniques for deploying and managing large-scale virtualized environments
  - **Dynamic resizing** for vertical scaling
    - Scale VMs up or down
  - **Live migration** of VMs
    - Move VMs between different physical machines (or even data centers) with shutting them down, ensuring operation during migration

VM dynamic resizing usato per lo scaling VERTICALE

è sempre possibile, spegnere una VM, riconfigurarla e poi riavviarla. ma non è sempre conveniente

- Fine-grain mechanism for adjusting VM resources (CPU, memory) on-the-fly
  - Example: app running on a VM consumes excessive resources, thus VM runs out of RAM and CPU
  - Solution: dynamically resize VM (aka *warm* resizing) to allocate more CPU and memory

✓ More cost-effective and faster than rebooting the VM

✗ Not all virtualization products and guest Oss support it

- What can be resized *without* stopping and rebooting the VM?
  - Number of virtual CPUs
  - Memory

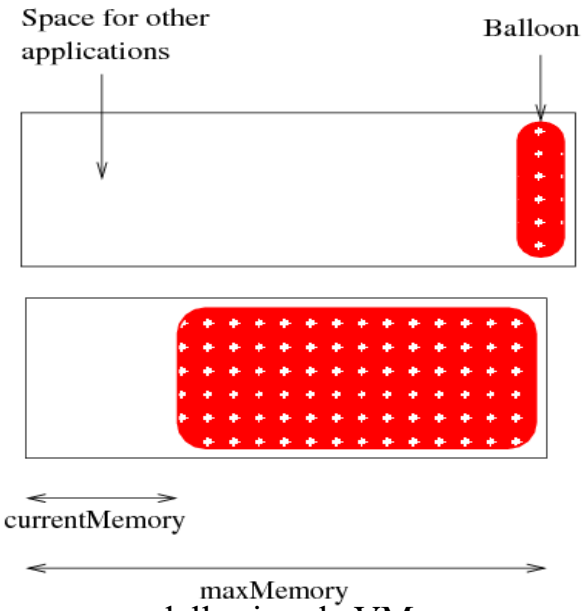
VM dynamic resizing: CPU

- Add or remove virtual CPUs (without turning off VM)
- Linux supports CPU hot-plug/hot-unplug
  https://www.kernel.org/doc/html/latest/core-api/cpu_hotplug.html
  - Relies on virtual file system `sysfs` (processor info is in `/sys/devices/system/cpu`)
    - `/sys/devices/system/cpu/cpuX` for cpuX (X = 0, 1, 2, …)
  - To turn on cpu #5:
    `echo 1 > /sys/devices/system/cpu/cpu5/online` direttamente da shell
  - To turn off cpu #5:
    `echo 0 > /sys/devices/system/cpu/cpu5/online`
- Manage VM CPU resizing using `virsh` letteralmente virtualshell
  - `virsh`: command line tool to configure and manage VMs, available with some hypervisors (KVM, Xen)
  - E.g., to set the number of vCPUs

```
virsh setvcpus <vm_name> <vcpu_count> --current
```


VM dynamic resizing: memory

- Based on *memory ballooning*
  - Mechanism used by hypervisors (e.g., KVM, Xen, VMware) to pass memory back and forth between hypervisor and guest OSs
- Balloon inflation: hypervisor takes memory from the VM
  - May swap to disk
- Balloon deflation: more memory is allocated to the VM when needed
  - Cannot exceed maxMemory limit



della singola VM

passiamo alla migrazione

VM migration: pros and cons

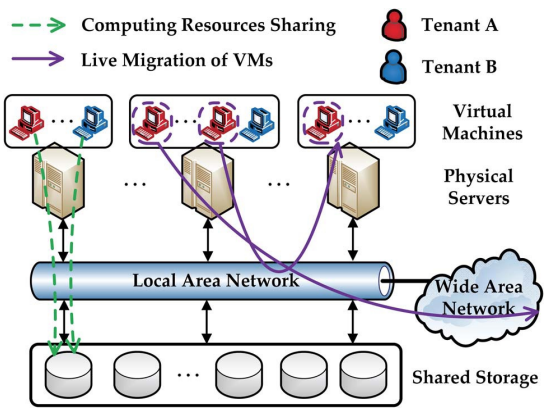
- Useful in clusters and virtual data centers to:
  - ✓ Consolidate infrastructure
  - ✓ Enable failover flexibility
  - ✓ Balance load
  - ✓ Improve maintenance

- Cons
  - ✓ Requires VMM support
  - ✓ Migration overhead ovvero l'aumento del downtime, dato che spengo la VM
  - ✓ WAN migration limited

non consideremo il caso di migrazione tra data center differenti

VM migration

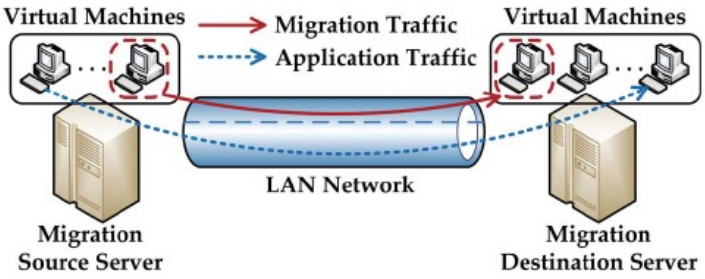
- Approaches to migrate VM instances:
  - Cold migration: shutdown the source VM, transfer its image to to the destination host, then restart it
    - ✗ VM downtime can be too long il downtime, comprende lo shutdown, il trasferimento e il boot della VM. (una VM è grande svariati GB)
    - ✗ VM image can be large and network bandwidth limited
  - **Live migration**: the source VM is running during migration
    - ✓ Minimal downtime
    - ✓ Widely used by large-scale environments (e.g., Google: > 1M migrations per month)



VM live migration

- Preliminary steps before VM live migration
  - Setup phase: determine source host, destination host, and VM to migrate (goals: load balancing, energy efficiency, server consolidation) by means of a selection algorithm
- What to migrate? Memory, storage, network connections
- How to migrate? Transparent migration for applications running inside the VM
  - Challenges: full migration transparency is difficult, application downtime is still possible: how to limit it?

qui entra in gioco il MAPE



VM live migration: storage

- We consider VM migration within a cluster environment

- To migrate storage
  - With shared storage
    - Use a SAN (Storage Area Network), NAS (Network Attached Server), or distributed file system (e.g., HDFS, Ceph) to store the VM disk image
  - Without shared storage
    - The source VMM stores the VM disk data on the source host local storage and the disk image is copied to the destination host

fisico o virtuale

occorre quindi aspettare il tempo per la copia

VM live migration: network

- To migrate network connections
  - Source VM has a virtual IP address and virtual MAC address, which can be distinct from the IP address and MAC address of source host
    - VMM maintains a mapping between virtual IP addresses and MAC addresses to their corresponding VMs
  - If source and destination hosts on a single switched LAN, network migration happen transparently grazie al protocollo ARP
    - The source host sends an unsolicited ARP reply to notify the network that the VM's IP address has moved to a new location (the destination host)
    - Possible packet loss limited to a few in-flight packets inteso come delay
  - Otherwise, various alternatives:
    - Use network layer forwarding tunneling
    - Update DNS records (if VM uses a domain name)
    - Use a load balancer in front of the VM

le prime due funzionano anche in data center sparsi geograficamente

debbo però aggiornare anche i proxy

VM live migration: memory

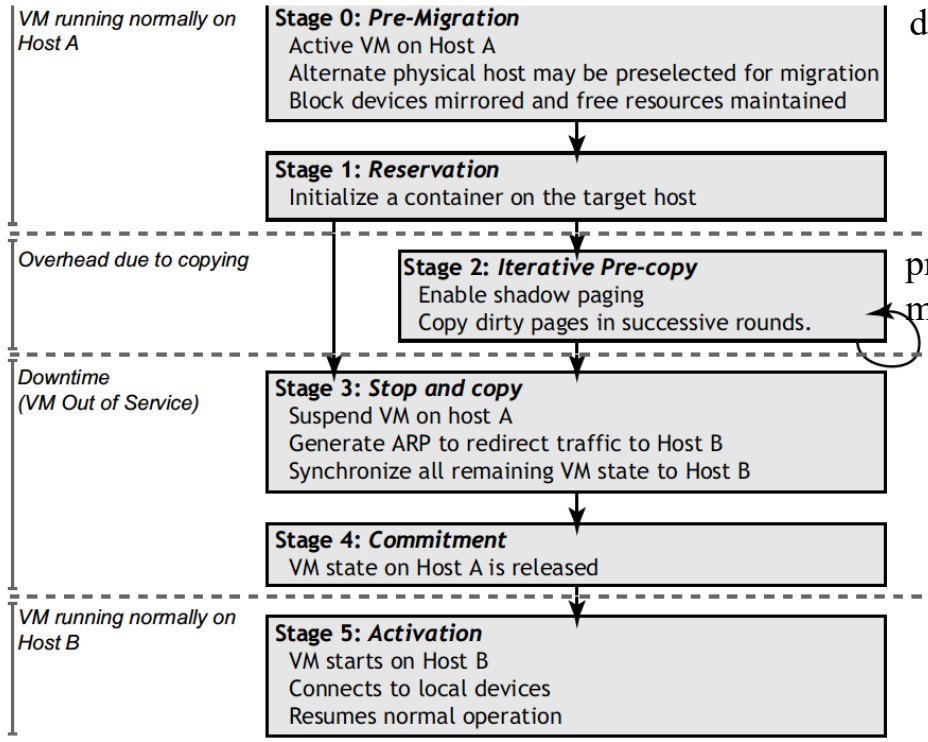
include anche i registri della CPU e dei driver

- To migrate memory (including CPU and device state):
  1. Pre-copy phase: VMM iteratively copies the memory pages from source VM to destination VM while source VM is running
     - During iteration *n* those pages dirtied during iteration *n*-1 are copied
  2. Stop-and-copy phase: source VM is suspended and the last dirty pages are copied, along with CPU state and device driver state; VM applications do not run
     - Downtime: can range from msec to sec, depending on memory size, application memory workload, network bandwidth
  3. Commitment and reactivation phases: destination VM is activated and recovers application execution; source VM is removed (and source host may be turned off)
- Known as **pre-copy** approach
  - Memory is copied *before* VM execution resumes at destination
  - Most commonly used technique, e.g., by KVM, VMWare, Xen, Google CE

n è arbitrario, oppure si usa un timer di esecuzione

il downtime è influenzato molto da applicazioni memory-intensive

VM live migration: pre-copy



dettagli del processo

prima storage, poi memory

Clark et al., Live Migration of Virtual Machines, NSDI 2005
https://www.usenix.org/legacy/event/nsdi05/tech/full_papers/clark/clark_html/

VM live migration: alternatives for memory

- Pre-copy struggles with memory-intensive apps
  - Especially write-intensive ones, because it cannot migrate memory faster than the rate at which memory is dirtied by running apps
- Two alternative approaches
  - Post-copy
  - Hybrid
- **Post-copy** approccio ottimale per applicazioni che sono memory-intensive
  - CPU state and device state: immediately transferred
  - VM execution control: transferred to destination host
  - Memory transfer: fetched on-demand as needed by the running VM (*pull* approach) le pagine arrivano tramite pull
  - ✓ Reduces downtime and overall migration time
  - ✗ Incurs app degradation due to page faults that must be resolved over the network i page fault sono più costosi

pessimo per applicazioni read-intensive

VM live migration: alternatives for memory

- **Hybrid**
  - Combines pre-copy and post-copy to balance migration efficiency and minimize application downtime
  - Special case of post-copy migration: post-copy preceded by a bounded pre-copy stage
  - Idea: before switching VM execution to destination host, transfer a subset of the most frequently accessed memory pages, so to reduce app performance degradation by minimizing the number of page faults after VM is resumed
  - ✓ Pre-copy reduces the number of future network-bound page faults

- Pre-copy still dominates in terms of support and usage
  - `virsh migrate` command

VM live migration: alternatives for memory

figura d'esempio

